

What Inference Is, and Why It Became Everyone's Problem

Inference Fundamentals, Part 1A: the on-ramp for the rest of the series

This series explains how large language models are served in production, for engineers and architects who work with these systems but have not built one. This first article is the shortest and the least technical. It answers three questions that the rest of the series assumes you have already settled: what inference actually is, why it turned from a research detail into an infrastructure discipline, and what changed when the weights of strong models became something you could download. Part 1B takes the physics from there.

Where does the time go? For more than a decade that was my question, asked of large data platforms on behalf of enterprises: was a query reading, computing, or waiting for a slot; was the compute sized for the concurrency the system actually saw or the concurrency someone guessed; what deserved a cache; and what did all of it cost per query, in terms finance could act on. More recently the question has moved to language models: agentic systems, and the open-source and frontier models beneath them. I came to inference expecting a new set of rules. What I found was the same discipline in different units: two hardware limits, a handful of formulas connecting them to what a user feels, and the familiar problem of which lever a given symptom is asking for. This series is for readers with that background who want to understand inference well enough to run it.

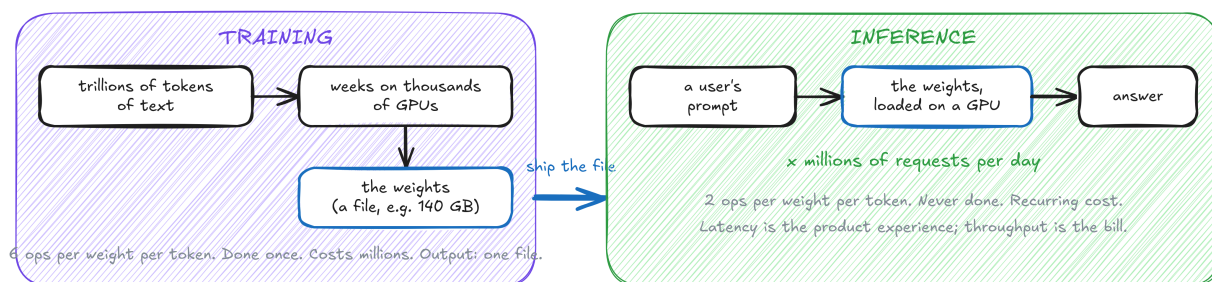
Training happens once. Inference happens forever.

A language model has two lives.

The first is training. A lab takes trillions of tokens of text, runs them through the model for weeks on thousands of GPUs, and adjusts the model's parameters, its weights, until it predicts text well. Training is enormously expensive, but it is done once, by the model's maker, and its output is a single file: the weights. For a 70-billion-parameter model that file is about 140 gigabytes at full precision.

The second life is inference. Someone loads the weights onto a GPU, sends in a prompt, and reads out an answer. Then they do it again for the next user, and the next, for as long as the product exists. Inference is cheap per request and never finishes. It is the recurring cost, and it is also the entire product experience: the pause before an assistant starts typing, the pace at which it types, the moment a voice agent answers or hesitates. All of that is inference.

Training happens once. Inference happens on every request, forever.



A model is trained once by its maker. It is then run, by anyone who has the weights, on every request for as long as the product exists.

There is a precise version of "cheap per request" that the rest of the series builds on. Processing one token during inference costs about two arithmetic operations per weight: one multiply, one add. Training costs about six, because each weight is used three times per token (once

forward, twice backward). So a single inference request on a 70B model is roughly 140 billion operations per token, which sounds enormous until you learn that an H100 GPU does about a thousand trillion operations per second. The catch, and the subject of Part 1B, is that arithmetic is not the bottleneck for most of a request. Memory is.

Why inference became an infrastructure discipline

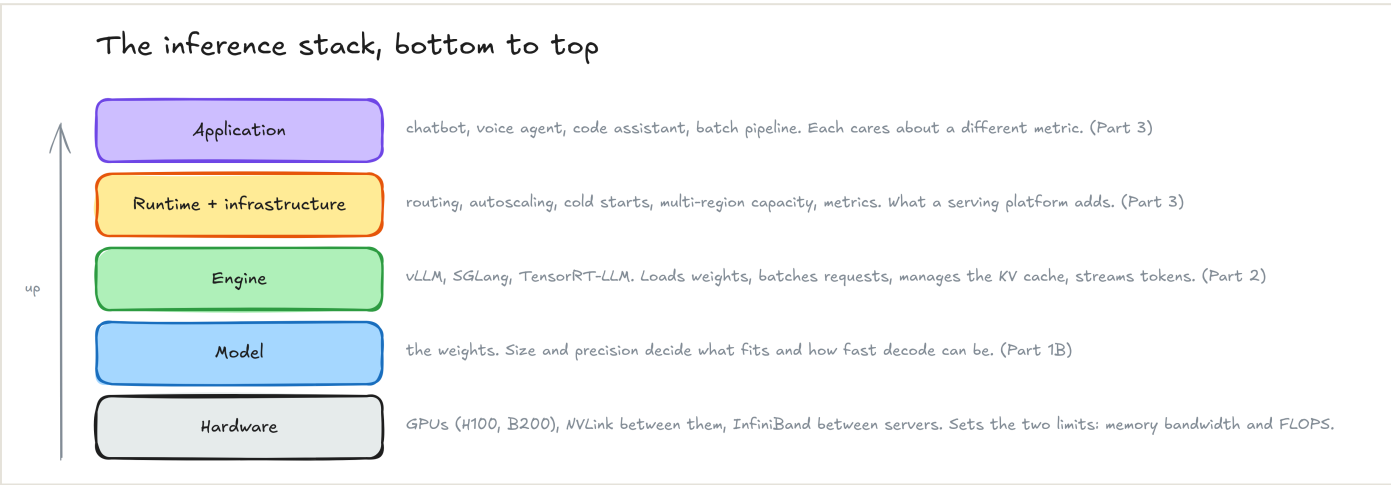
For most of the history of machine learning, serving a model was an afterthought. Models were small enough to run on a CPU, requests were independent, and "deployment" meant wrapping a function in a web server. Three things changed that.

The models got big enough that hardware became the design constraint. A 70B model does not fit on a single 80 GB GPU at full precision. Its per-token work is bounded by how fast the GPU can read its own memory, not by how fast it can compute. Serving such a model well means understanding the GPU's two limits, memory bandwidth and arithmetic rate, and which one each phase of a request hits. That is engineering, not configuration.

The workloads got latency-sensitive and bursty. A batch prediction job can run overnight. A chat assistant has a human waiting, and the human notices a half-second pause. A voice agent has a few hundred milliseconds before silence becomes awkward. Traffic arrives in waves, with a lunchtime peak that is ten times the overnight floor. Serving these workloads means autoscaling GPUs that take a minute to start, holding headroom for ramps, and choosing, for every workload, how much latency to trade for how much throughput.

The cost became visible. When a request costs a fraction of a cent, nobody counts. When a product serves a hundred million tokens a day and GPUs cost dollars per hour, the CFO counts. Cost per million tokens became a number that finance teams compare across vendors, and it is set almost entirely by how well the serving system batches work.

Put those together and inference stopped being a step in a pipeline and became a stack.



At the bottom is hardware: GPUs, the links between them, the links between servers. Above it is the model: its size and precision decide what fits and how fast it can run. Above that is the engine, software like vLLM, SGLang or TensorRT-LLM, which loads the weights, groups requests together, manages memory, and streams tokens back. Above that is the runtime and infrastructure layer that a serving platform adds: routing requests to the right replica, scaling replicas up and down, handling the cold start when a new one comes online, spreading capacity across clouds and regions, and measuring all of it. And at the top is the application, which is the reason any of this exists and which decides which of the numbers below it actually matter.

This series is a walk up that stack. Part 1B is the model and the hardware. Part 2 is the engine. Part 3 is the runtime, the metrics, and the application. Part 4 is the set of decisions that sit on top: quantization, structured outputs, tool calling, and the tail latency of agents.

What open-weight models changed

Until 2023, with few exceptions, if you wanted a strong language model you called an API. The model's maker ran the inference; you sent tokens and received tokens; the stack above was their problem. Then the weights of strong models started being published.

The turning points, roughly in order: Meta's Llama 2 in mid-2023 made a capable model available under a license that permitted commercial use. Mistral 7B, later that year, showed a small open model could be surprisingly strong. Llama 3 in April 2024 narrowed the gap to closed models on many tasks, and Alibaba's Qwen 2.5 family that September did the same with an emphasis on multilingual and coding work. DeepSeek V3 in December 2024 and R1 in January 2025 demonstrated that a mixture-of-experts model with 671 billion parameters (37 billion active per token) could be trained and released openly, under an MIT license, with reasoning capability that had previously been the preserve of closed labs. And in August 2025, OpenAI released its first open-weight models, gpt-oss-120b and gpt-oss-20b, under Apache 2.0. By the end of 2025, for many workloads there was an open-weight model within reach of the best closed one.

Two consequences follow, and they are the reason this series exists.

Three ways to run a model

Closed model behind an API

you send tokens, you get tokens
no infrastructure to run
no control over latency, cost per token, data path, or model version
the provider does all of Part 1B-3

Open-weight model on a managed platform

you pick the model and the GPU
platform runs engine, routing, autoscaling, multi-cloud capacity
you set the performance targets
this series is what the platform does

Open-weight model, self-hosted

you run everything: GPUs, engine, scaling, cold starts, on-call
maximum control, maximum burden
this series is what you must know

Open-weight models made the second and third options possible. That is why inference stopped being the model maker's problem and became everyone's.

First, **there are now three ways to run a model, not one.** You can still call a closed model's API and let the provider handle everything. You can take an open-weight model and run it on a managed inference platform, which supplies the engine, the routing, the autoscaling and the multi-cloud capacity, while you choose the model, the GPU, and the performance targets. Or you can self-host, which gives maximum control and maximum burden. The second and third options did not exist in a practical form before open weights. They are also where the interesting decisions live, because in both cases someone has to decide what latency to promise, what concurrency to run, what to quantize, and when to add a GPU.

Second, **the decisions became yours.** With a closed API, the provider chose the tradeoffs and you lived with them. With open weights, the same model can be served for a voice agent that needs a first token in 200 milliseconds or for a nightly batch job that cares only about cost per token, and the configurations for those two are nearly opposite. Whoever runs the model, whether that is your team or a platform acting on your instructions, has to understand the tradeoffs well enough to place each workload correctly. That understanding is what Parts 1B through 4 are for.

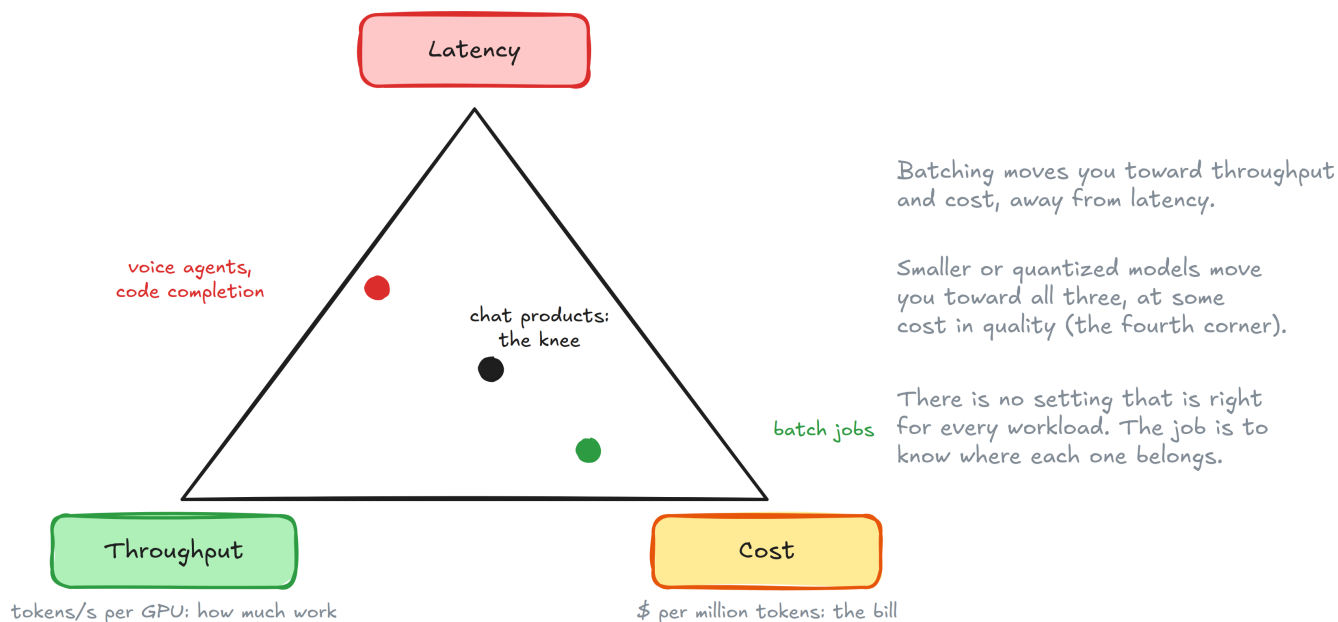
There is a third, quieter consequence. When the model is a file anyone can download, the model itself stops being the competitive advantage. Serving it well does: faster, cheaper, more reliably, at the latency the product needs. That is why a whole category of inference platforms and a whole job family of forward-deployed and solutions engineers grew up around open-weight models in the space of two years.

The triangle every decision lives in

If there is one picture to carry from this article into the rest of the series, it is this one.

Every inference decision is a position inside this triangle

TTFT, ITL: what one user feels



Every inference system is trying to do three things at once: respond quickly to each user (latency), serve as many tokens per second per GPU as possible (throughput), and spend as little per token as possible (cost). The three pull against each other. The main lever, batching, which means serving many requests in the same pass through the model, moves you toward throughput and cost and away from latency. A smaller or lower-precision model moves you toward all three at once, but at some cost in output quality, which is the triangle's hidden fourth corner.

Different applications belong at different points. A voice agent or a code-completion tool sits near the latency corner and accepts a higher cost per token. A batch summarization job sits at the throughput-and-cost edge and does not care about latency at all. A chat product sits in the middle, at a point Part 1B will call the knee, where batching is still nearly free and latency is still comfortable. There is no setting that is right for every workload. The job of anyone running inference is to know where each workload belongs and to put it there.

How to read the rest of the series

Part 1B, The Physics of an LLM Request, explains why the triangle has the shape it does. A request runs in two phases with different limits, and once you see that, you can derive the shape of every benchmark curve from a GPU's spec sheet.

Part 2, What the Serving Engine Does with the Physics, covers the software: how requests are batched, how memory is managed, and every optimization in the field classified by which limit it attacks.

Part 3, Reading Benchmarks and Running the System, is for operators: what each metric is made of, why the 99th percentile misbehaves, how to convert tokens into dollars, and how an autoscaler is really a statement about cold starts.

Part 4, Quantization, Structured Outputs and Tool Calling, from the Physics, applies the model to the decisions customers actually ask about: the roofline and its ridge point, which quantization format helps which phase, how a JSON schema becomes a token mask, why an agent's first token gets slower every turn, and why one user action in ten hits the tail when every model call is fast.

Each part ends with a few questions to check yourself against. Here are three for this one.

Check yourself

- 1. Why is inference, not training, the cost that matters for a product?** Training is paid once by the model's maker and produces a file. Inference is paid on every request for as long as the product exists, so it is the recurring cost, and it is also what the user actually experiences.
 - 2. What did open-weight models change about who makes serving decisions?** With a closed API the provider chose the latency, throughput and cost tradeoffs. With open weights, the same model can be configured very differently for different workloads, and whoever runs it, a team or a platform on the team's behalf, has to make those choices.
 - 3. Why is there no single right configuration for a model?** Because latency, throughput and cost pull against each other, and different applications need different corners of that triangle. A voice agent and a batch job served from the same weights want nearly opposite settings.
-

Part 1B begins with tokens, weights, and the two operations per weight that everything else is built on.

Radx Radhakrishnan works on LLM inference and agentic systems, after a decade of performance work on enterprise data platforms.

Sources

- Meta, "Meta and Microsoft Introduce the Next Generation of Llama," July 18, 2023. <https://about.fb.com/news/2023/07/llama-2/>
- Mistral AI, "Announcing Mistral 7B," September 27, 2023. <https://mistral.ai/news/announcing-mistral-7b>
- Meta, "Introducing Meta Llama 3," April 18, 2024. <https://ai.meta.com/blog/meta-llama-3/>
- Qwen Team, "Qwen2.5: A Party of Foundation Models!," September 19, 2024. <https://qwenlm.github.io/blog/qwen2.5/>
- DeepSeek, "Introducing DeepSeek-V3," December 26, 2024. <https://api-docs.deepseek.com/news/news1226>
- DeepSeek, "DeepSeek-R1 Release," January 20, 2025. <https://api-docs.deepseek.com/news/news250120>
- OpenAI, "Introducing gpt-oss," August 5, 2025. <https://openai.com/index/introducing-gpt-oss/>
- Kaplan et al., "Scaling Laws for Neural Language Models," 2020, Section 2.1 (the 6N per-token training-compute estimate). <https://arxiv.org/abs/2001.08361>
- NVIDIA, H100 Tensor Core GPU product page (989 TFLOPS FP16 dense; 1,979 with sparsity). <https://www.nvidia.com/en-us/data-center/h100/>